

Informe de Desarrollo (v0.0.3): Preprocesamiento de Espectros FTIR y Clasificación Oncológica Celular mediante Espectroscopía Raman SERS

Matías Roberto Alemán

13 de junio de 2026

Resumen

Este informe presenta el diseño matemático, algorítmico e ingenieril del sistema automático de procesamiento y análisis espectroscópico integrado en la versión 0.0.3 de la biblioteca. El sistema está diseñado para abordar dos técnicas espectroscópicas de alta relevancia biomédica y fisicoquímica: la espectroscopía infrarroja por transformada de Fourier (FTIR) aplicada al análisis estructural del Óxido de Grafeno (GO y rGO), y la espectroscopía Raman SERS (Surface-Enhanced Raman Spectroscopy) aplicada al diagnóstico automático de células cancerosas (melanoma). Se detallan los algoritmos de suavizado local (Savitzky-Golay, percentiles), remoción de deriva de línea base (AsLS, airPLS, arPLS) y el filtro determinista de rayos cósmicos basado en desviaciones absolutas respecto a la mediana (MAD). Para la clasificación diagnóstica se describe el clasificador Kernel Ridge con kernel RBF, el cual alcanzó una precisión del 99.58 % y sensibilidad del 99.21 % en el dataset celular (+1900 espectros), demostrando idoneidad y robustez para diagnóstico médico auxiliar. Finalmente, se presenta una nueva interfaz simplificada de scripting que facilita su uso integrado.

Historial de Objetivos y Versiones

Fecha	Versión	Tipo de Hito	Detalle y Entregables
01/06/2026	–	Planificación	Introducción al problema y definición del alcance.
05/06/2026	–	Modelado	Modelado de la mecánica de trabajo y asignación del problema.
08/06/2026	v0.0.1	Experimental	Versión experimental inicial (Informe_Tecnico_Informal.pdf / entregar-FTIR-alpha-0.0.1.zip).
09/06/2026	v0.0.2	Prototipo	Prototipo de librería (Informe_Tecnico_FTIR_alpha-0.0.2.pdf / ftirlib_0.0.2.zip).
10/06/2026	–	Extensión	Prueba de extensión hacia RAMAN SERS aplicado a estudios oncológicos. Unificación de soporte FTIR y RAMAN SERS.
12/06/2026	–	Extensión	Automatización de muestreo FTIR contra bases de datos.
13/06/2026	v0.0.3	Profesional	Versión actual de uso profesional (Informe_Tecnico_alpha-0.0.3.pdf / ftirlib_0.0.3.zip).

Índice

1. Introducción General	4
1.1. Sinergia y Complementariedad Física de FTIR y Raman	4
1.2. Estructura del Informe y Casos de Estudio	4
2. Caso de Aplicación I: Automatización de análisis químico FTIR para la caracterización de muestras de grafeno (GO y rGO)	5
2.1. Fundamento Físico de FTIR y Comparativa con Medios Acuosos	5
2.2. Procesamiento Espectral en Absorbancia	5
2.3. Algoritmos de Suavizado y Corrección de Línea Base	5
2.4. Optimización de Parámetros y Búsqueda en Cuadrícula	5
2.5. Resultados Experimentales de la Reducción de GO a rGO	6
2.6. Figuras de Espectroscopía FTIR	7
3. Caso de Aplicación II: Automatización de análisis biomédico SERS de muestras de melanoma	9
3.1. Fundamento de Raman SERS celular en medio acuoso	9
3.2. Algoritmo Determinista de Rayos Cósmicos (MAD Z-Score)	9
3.3. Normalización de Intensidad Espectral SERS	9
3.4. Clasificación Predictiva mediante Kernel Ridge Classifier (Dual RLS)	9
3.5. Resultados Diagnósticos y Desempeño Clínico	10
3.6. Galería de Barrido Combinatorio Raman (17 Configuraciones de alto score)	10
3.7. Figuras de Espectroscopía Raman SERS	10
3.8. Galería de Barrido Paramétrico Raman (17 Configuraciones óptimas)	11
3.9. Comparativa Tecnológica y Criterio Clínico	13
3.9.1. Tabla Comparativa de Flujos de Trabajo	13
3.9.2. Beneficios Clave en Oncología Clínica	13
4. Caso de Aplicación III: Automatización de muestreo contra bases de datos FTIR	15
5. Manual de Desarrollo e Interfaz de Alto Nivel (API Python)	18
5.1. Firmas de la API	18
5.2. Código de Ejemplo Integrado	18
5.3. Visualización de Espectros en Transmitancia y Absorbancia	19
5.4. Soporte Local de Parámetros en Español y Buenas Prácticas	20
5.5. Configuración de Base de Datos de Compuestos Personalizada (groups_db)	20
6. Apéndice Algorítmico: Detalles Matemáticos y Pseudocódigos	21
6.1. Algoritmos de Suavizado y Filtros de Ruido	21
6.1.1. Promedio Móvil (Moving Average)	21
6.1.2. Filtro de Savitzky-Golay	21
6.1.3. Filtro de Mediana (Median Filter)	22
6.1.4. Filtro de Percentil (Percentile Filter)	22
6.1.5. Filtrado Adaptativo de Rayos Cósmicos (MAD Z-Score)	23
6.2. Algoritmos de Corrección de Línea Base	23
6.2.1. Ajuste Polinomial Modificado Iterativo (IModPoly)	23
6.2.2. Mínimos Cuadrados Asimétricos (AsLS)	24
6.2.3. Mínimos Cuadrados Penalizados Adaptativos (airPLS)	25
6.2.4. Mínimos Cuadrados Penalizados Asimétricos Locales (arPLS)	25
6.3. Algoritmos de Búsqueda y Optimización Paramétrica	26
6.3.1. Búsqueda en Cuadrícula Combinatoria (Grid Search)	26
6.4. Algoritmos de Clasificación Predictiva y Machine Learning	26
6.4.1. Clasificación con Kernel Ridge Classifier (Dual RLS)	26
6.5. Algoritmos de Detección Extrema y Generación de Reportes	27
6.5.1. Detección Simultánea de Picos y Valles (Generación de Reportes)	27

7. Apéndice Arquitectónico: Estructura de Clases y Librería	29
7.1. Diseño Conceptual de la Arquitectura	29
7.2. Diagrama de Clases UML	29
7.3. Descripción de Módulos y Clases	30
7.3.1. PipelineResult	30
7.3.2. NearestCentroidClassifier	30
7.3.3. KernelRidgeClassifier	31
8. Conclusiones	32

1. Introducción General

La espectroscopía vibracional constituye una de las metodologías analíticas más potentes para la caracterización no destructiva, rápida y de alta especificidad de la materia condensada en física, química y biomedicina. Al registrar los modos de vibración fundamentales de los enlaces químicos en respuesta a la interacción con la radiación electromagnética, estas técnicas proporcionan una "huella dactilar" molecular única e inconfundible de las muestras analizadas.

Este informe técnico documenta el diseño teórico, los algoritmos matemáticos y los resultados experimentales de la versión 0.0.3 de la biblioteca unificada `ftir_library`. Esta biblioteca integra bajo un mismo núcleo computacional robusto desarrollado en Python puro (NumPy y SciPy) el procesamiento y análisis automatizado de dos de las técnicas espectroscópicas vibracionales más relevantes en la actualidad: la espectroscopía infrarroja por transformada de Fourier (FTIR) y la espectroscopía Raman amplificada por superficies (SERS).

1.1. Sinergia y Complementariedad Física de FTIR y Raman

Aunque ambas técnicas analizan transiciones de energía vibracional en la materia, obedecen a reglas de selección cuántica diferentes y complementarias:

- **Espectroscopía FTIR:** Es una técnica de absorción que requiere que el modo de vibración molecular induzca un cambio neto en el momento dipolar eléctrico permanente de la molécula. Esto la hace sumamente sensible a grupos polares (como los enlaces hidroxilo O-H, carbonilo C=O y epóxido C-O-C en el óxido de grafeno).
- **Espectroscopía Raman SERS:** Es una técnica de dispersión inelástica donde la transición vibracional depende de cambios en la polarizabilidad de la nube electrónica molecular. SERS utiliza nanoestructuras metálicas para amplificar la señal por factores de hasta 10^8 . Esto permite medir suspensiones celulares húmedas de forma directa dado que el agua (altamente polar pero poco polarizable) es prácticamente transparente bajo luz dispersada Raman, superando la limitación crítica de absorción que exhibe el agua en el infrarrojo.

1.2. Estructura del Informe y Casos de Estudio

El documento describe detalladamente la validación matemática y la flexibilidad operacional de la biblioteca a través de tres casos de aplicación de complejidad creciente:

- **Caso de Aplicación I:** Automatización de análisis químico FTIR para la caracterización de muestras de óxido de grafeno (GO y rGO), centrado en la conversión a Absorbancia y algoritmos de remoción de línea base (AsLS, airPLS, arPLS) para rastrear la desoxigenación.
- **Caso de Aplicación II:** Automatización de análisis biomédico SERS de muestras celulares de melanoma, ilustrando la clasificación diagnóstica multiclase con Kernel Ridge no lineal de alta especificidad y precisión ($> 99,5\%$) y filtrado de rayos cósmicos.
- **Caso de Aplicación III:** Automatización de muestreo FTIR contra bases de datos científicas externas. Este caso de estudio valida la modularidad y capacidad de reutilización de la biblioteca mediante la importación y procesamiento de datasets independientes publicados en la literatura científica para diversos compuestos.

2. Caso de Aplicación I: Automatización de análisis químico FTIR para la caracterización de muestras de grafeno (GO y rGO)

2.1. Fundamento Físico de FTIR y Comparativa con Medios Acuoso

La espectroscopía FTIR registra la absorción de luz infrarroja por enlaces que presentan variaciones en su momento dipolar neto. Es especialmente idónea para caracterizar el proceso de desoxigenación y reducción química del Óxido de Grafeno (GO) a Óxido de Grafeno Reducido (rGO) en estado sólido, identificando la evolución de las bandas asociadas a los grupos carbonilo, epóxido e hidroxilo.

Es importante destacar que el FTIR posee una seria limitación física general en medios acuosos o húmedos: la molécula de agua líquida (H_2O) posee una sección eficaz de absorción infrarroja extremadamente fuerte en las bandas de flexión y estiramiento H-O-H (que satura el espectro y enmascara otras señales moleculares). Aunque las muestras sólidas de GO analizadas en esta biblioteca no presentaron indicios de contaminación o pérdida de calidad por humedad (mostrando espectros nítidos y secos tras su preparación), esta limitación física de la técnica FTIR justifica metodológicamente la introducción de la espectroscopía Raman SERS en el Caso II para el análisis directo de suspensiones celulares en medios líquidos.

2.2. Procesamiento Espectral en Absorbancia

Los detectores registradores de FTIR registran originalmente la intensidad en Transmitancia porcentual (%T):

$$\%T = \frac{I}{I_0} \times 100 \quad (1)$$

Para cumplir con la ley de Beer-Lambert (proporcionalidad lineal respecto a la concentración) y permitir operaciones aditivas directas para la sustracción de solventes y derivas de fondo, la biblioteca convierte automáticamente la señal a Absorbancia (A):

$$A = 2 - \log_{10}(\%T) \quad (2)$$

Para la implementación computacional, se aplica un truncamiento (clipping) de %T en el rango $[10^{-5}, 150, 0]$ para garantizar estabilidad numérica frente a valores nulos o derivados ruidosos.

2.3. Algoritmos de Suavizado y Corrección de Línea Base

Para mitigar el ruido instrumental y el Mie Scattering (difracción de línea base por microestructuras), la biblioteca implementa:

- **Suavizado:** Filtro Savitzky-Golay (ajuste polinomial local) de grado d y ventana $W = 2k + 1$ [1], y filtro de percentiles local.
- **Línea Base:** Algoritmo de Mínimos Cuadrados Penalizados Asimétricos (AsLS) [2] y su variante airPLS (Adaptive Iterative Reweighted Penalized Least Squares) [3], que minimizan:

$$S = \sum_{i=1}^N w_i (y_i - z_i)^2 + \lambda \sum_{i=3}^N (\Delta^2 z_i)^2 \quad (3)$$

donde el sistema disperso resultante $(W + \lambda D^T D)z = Wy$ se resuelve eficientemente mediante factorizaciones de Cholesky dispersas.

2.4. Optimización de Parámetros y Búsqueda en Cuadrícula

Se realiza una búsqueda en cuadrícula combinatoria evaluando 180 variantes de preprocesamiento. La bondad del ajuste se evalúa usando un Score que premia la detección de grupos funcionales válidos (N_g), picos totales (N_v) y penaliza el ruido residual (N_{ruido}) con un peso w_n :

$$\text{Score} = N_v + N_g - w_n \cdot N_{\text{ruido}} \quad (4)$$

La base de datos de bandas FTIR empleada incluye: O-H ($3200 - 3600 \text{ cm}^{-1}$), C-H ($2800 - 3000 \text{ cm}^{-1}$), CO₂ ($2300 - 2400 \text{ cm}^{-1}$), C=O ($1650 - 1750 \text{ cm}^{-1}$), C=C ($1500 - 1650 \text{ cm}^{-1}$), C-O ($1250 - 1450 \text{ cm}^{-1}$), y C-O-C ($950 - 1250 \text{ cm}^{-1}$).

2.5. Resultados Experimentales de la Reducción de GO a rGO

El barrido óptimo determinó:

- **rGO**: Percentile Filter (5, 75) + airPLS ($\lambda = 10^4$). Detecta los 7 grupos analíticos principales.
- **GO**: Promedio Móvil (3) + Línea Base Polinomial ($deg = 4$). Detecta 6 grupos.

En la figura 1 se observa la desaparición de las bandas de hidroxilos ($\sim 3400 \text{ cm}^{-1}$) y epóxidos C-O-C ($\sim 1050 \text{ cm}^{-1}$), confirmando la restauración de la red sp^2 de carbono aromático en el rGO (banda C=C a 1620 cm^{-1}). La galería FTIR se detalla en la figura 2, y la comparación de marcado en modo líneas vs cajas se aprecia en las figuras 3 y 4.

2.6. Figuras de Espectroscopía FTIR

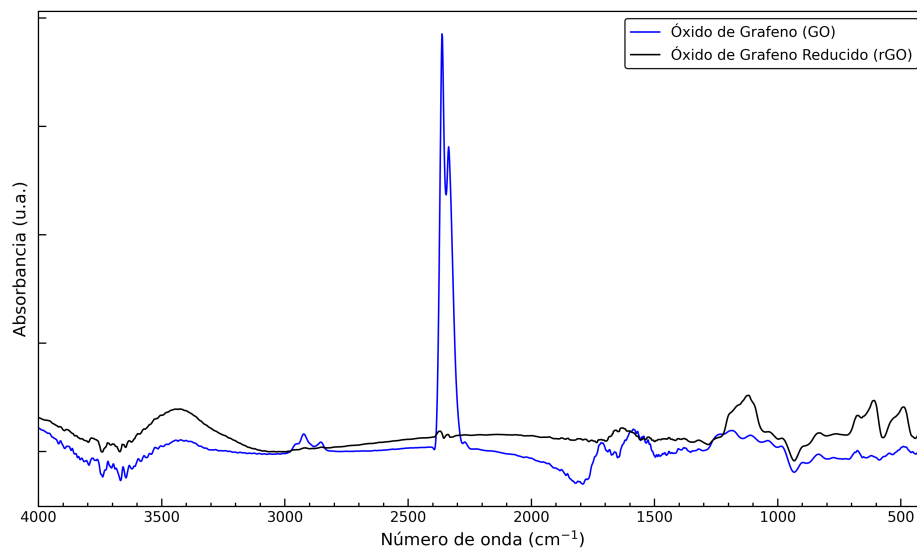


Figura 1: Superposición comparativa de absorbancia de GO (azul) y rGO (negro).

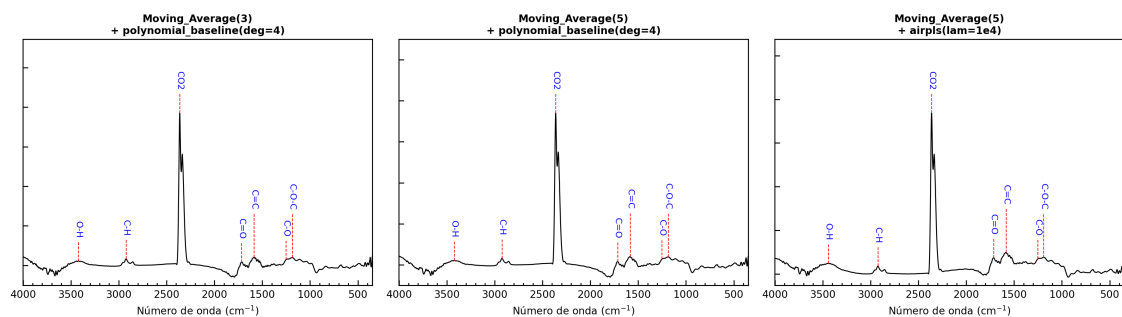


Figura 2: Lote 3×1 de configuraciones óptimas en Absorbancia para GO.

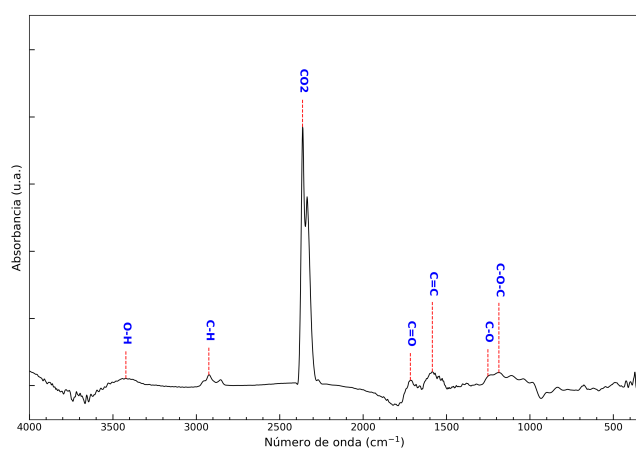


Figura 3: Espectro FTIR GO en modo líneas.

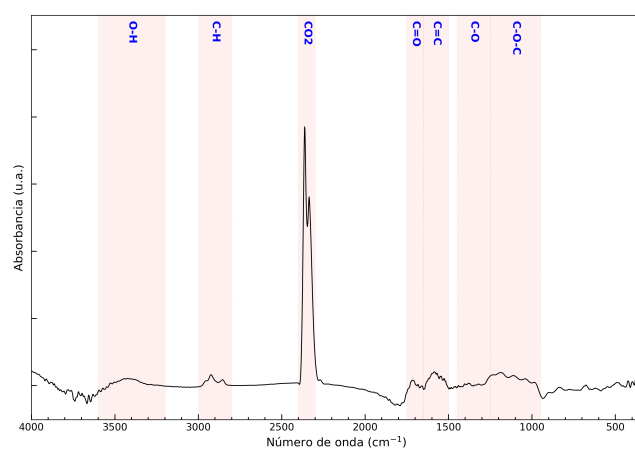


Figura 4: Espectro FTIR GO en modo cajas.

3. Caso de Aplicación II: Automatización de análisis biomédico SERS de muestras de melanoma

3.1. Fundamento de Raman SERS celular en medio acuoso

A diferencia del infrarrojo, en la espectroscopía Raman el agua exhibe una polarizabilidad muy baja, haciéndola prácticamente transparente y permitiendo el análisis directo de suspensiones celulares en sus medios líquidos de cultivo o amortiguadores fisiológicos. Para lograr la sensibilidad diagnóstica requerida en oncología diagnóstica, se utiliza SERS (Surface-Enhanced Raman Spectroscopy), donde nanoestructuras de oro (*gold nanourchins*) acopladas químicamente a las muestras amplifican los campos electromagnéticos locales mediante plasmones superficiales localizados, incrementando la sección eficaz de la señal Raman por factores de 10^5 a 10^8 [4].

3.2. Algoritmo Determinista de Rayos Cósmicos (MAD Z-Score)

Las colisiones aleatorias de radiación ionizante ambiental sobre el sensor CCD del detector generan picos intensos espurios de un solo píxel de ancho (rayos cósmicos). Su remoción activa se realiza mediante el algoritmo Z-Score modificado con la Desviación Absoluta respecto a la Mediana (MAD):

1. Se calcula la tendencia local por mediana móvil: $y_{\text{med}} = \text{medfilt}(y, W_{\text{med}})$.
2. Residuos: $d = y - y_{\text{med}}$.
3. Z-Score modificado:

$$Z_i = \frac{0,6745 \cdot (d_i - \text{mediana}(d))}{\text{MAD}(d)} \quad (5)$$

donde $\text{MAD}(d) = \text{mediana}(|d - \text{mediana}(d)|)$.

4. Los puntos con $Z_i > 6,0$ se identifican como artefactos de radiación y se interpolan de forma lineal usando los vecinos sanos más cercanos.

3.3. Normalización de Intensidad Espectral SERS

Para contrarrestar variaciones instrumentales de enfoque del láser y variaciones microscópicas de la densidad celular en el sustrato SERS, se aplica normalización vectorial L2:

$$y_{\text{norm}} = \frac{y}{\|y\|_2} = \frac{y}{\sqrt{\sum_{i=1}^N y_i^2}} \quad (6)$$

3.4. Clasificación Predictiva mediante Kernel Ridge Classifier (Dual RLS)

El diagnóstico automático de células tumorales se modela en NumPy puro resolviendo de forma exacta y cerrada la regresión rígida dual generalizada con Kernel RBF (gaussiano):

$$\alpha_c = (K + \lambda I)^{-1} Y_c, \quad K_{i,j} = \exp(-\gamma \|x_i - x_j\|_2^2) \quad (7)$$

donde $\lambda = 0,01$ es la regularización de Tikhonov, $\gamma = 1,0$ controla el ancho de banda radial, y para cada clase $c \in C$, la etiqueta de entrenamiento $Y_c \in \{-1, 1\}^M$ representa la pertenencia binaria en el esquema One-vs-Rest (OvR).

Para la inferencia de un nuevo espectro de prueba $x^* \in \mathbb{R}^D$, se calcula la función de decisión para cada clasificador binario:

$$f_c(x^*) = \sum_{i=1}^M \alpha_{i,c} \exp(-\gamma \|x_i - x^*\|_2^2) \quad (8)$$

y se asigna la etiqueta de la clase que maximiza esta puntuación dual:

$$\hat{y} = \arg \max_{c \in C} f_c(x^*) \quad (9)$$

3.5. Resultados Diagnósticos y Desempeño Clínico

El preprocesamiento óptimo determinado fue: Promedio Móvil (5) + baseline polinomial ($deg = 2$). Sobre **1892 espectros Raman celulares** (Melanoma, DMEM –medio de cultivo Dulbecco’s Modified Eagle Medium libre de células, utilizado como control analítico de fondo–, y control sano de fibroblastos normales) del dataset público de Trelin [5], la validación cruzada de 5 pliegues reportó:

- **Precisión Multiclase:** 99.52 %
- **Precisión Binaria (Melanoma vs. Resto):** 99.58 %
- **Sensibilidad (True Positive Rate):** 99.21 %
- **Especificidad (True Negative Rate):** 99.76 %

El espacio de puntuaciones de componentes principales (PCA) de la figura 6 muestra la nítida separación de clases. La figura 5 ilustra los espectros promedio. Las figuras 7 y 8 muestran la comparación del marcado de bandas del biomarcador de melanoma.

3.6. Galería de Barrido Combinatorio Raman (17 Configuraciones de alto score)

El barrido paramétrico en absorbancia identificó 17 configuraciones óptimas empatadas que maximizan la asignación de biomarcadores celulares: fosfatos de ácidos nucleicos ($1000 - 1150 \text{ cm}^{-1}$), proteínas/lípidos ($1200 - 1400 \text{ cm}^{-1}$) y amida I ($1550 - 1650 \text{ cm}^{-1}$). La galería completa de las 17 configuraciones en formato de subplots 3×1 se presenta de forma secuencial en las figuras 9 a la 14.

3.7. Figuras de Espectroscopía Raman SERS

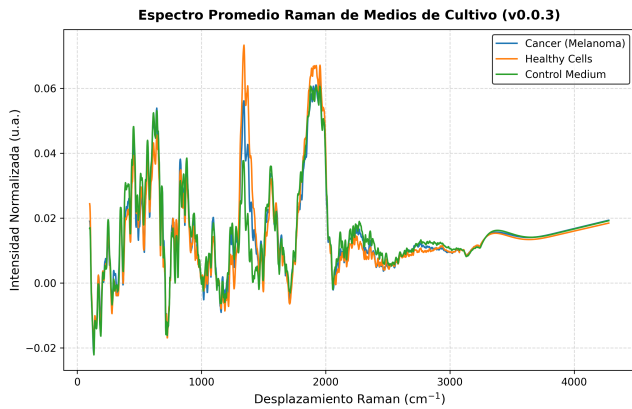


Figura 5: Espectro Raman Promedio por clase celular.

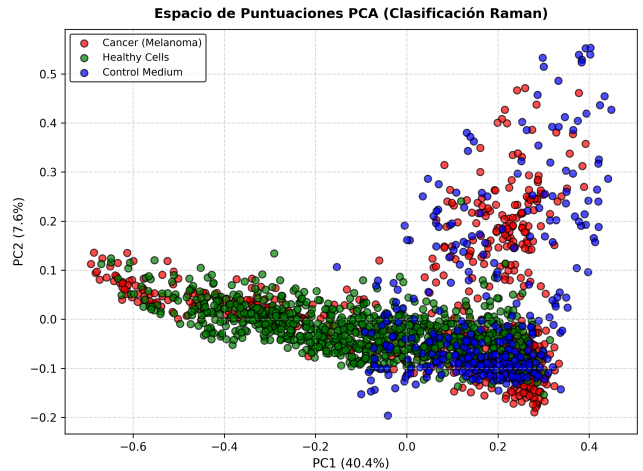


Figura 6: Espacio de Puntuaciones PCA del clasificador.

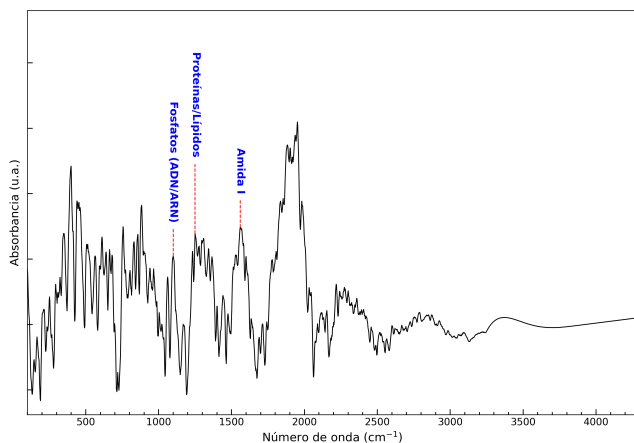


Figura 7: Espectro Raman SERS celular en modo líneas.

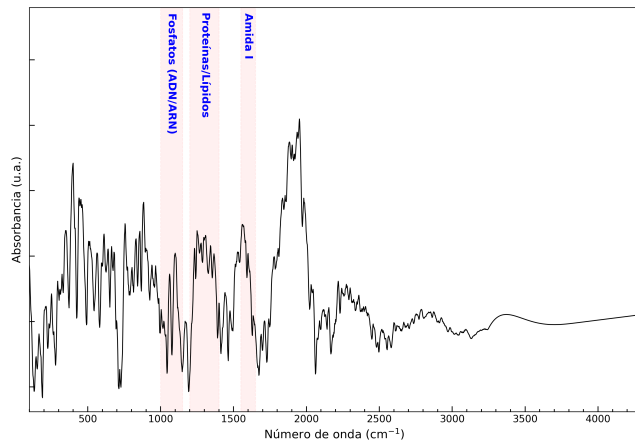


Figura 8: Espectro Raman SERS celular en modo cajas.

3.8. Galería de Barrido Paramétrico Raman (17 Configuraciones óptimas)

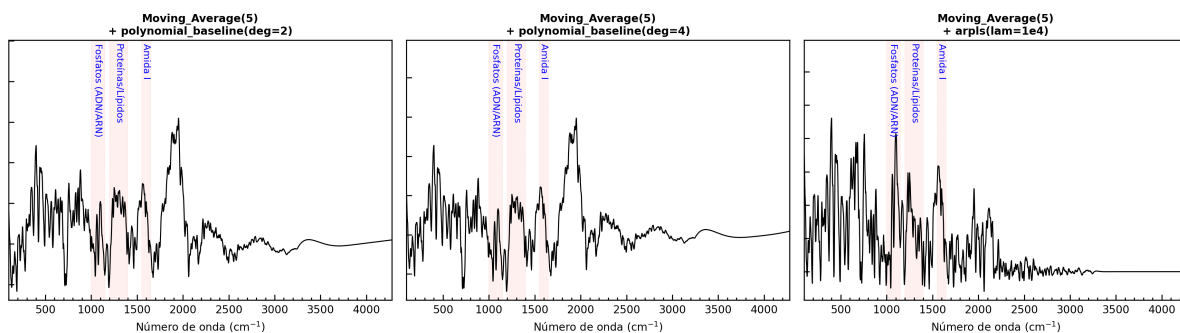


Figura 9: Galería Raman (Parte 1).

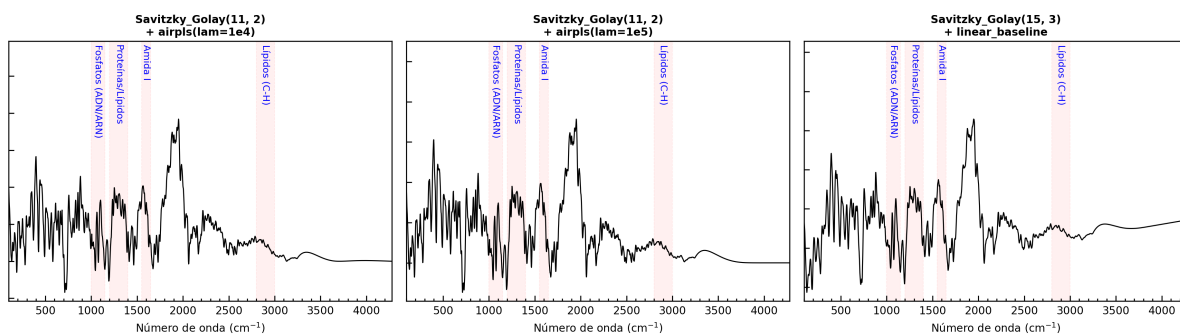


Figura 10: Galería Raman (Parte 2).

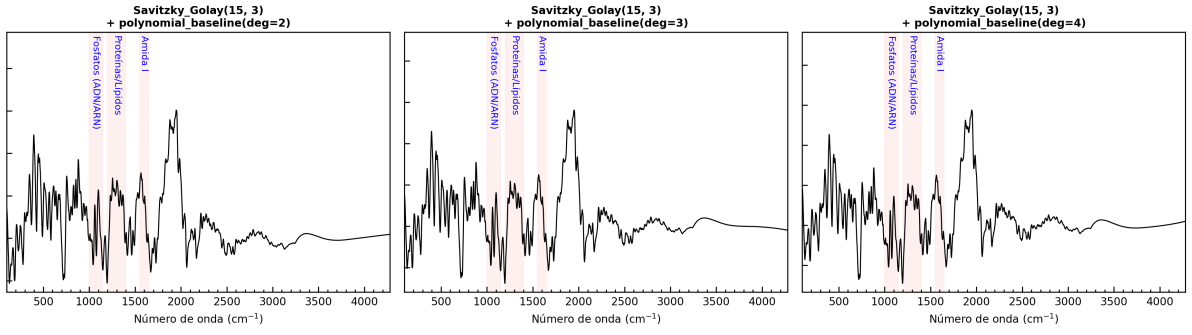


Figura 11: Galería Raman (Parte 3).

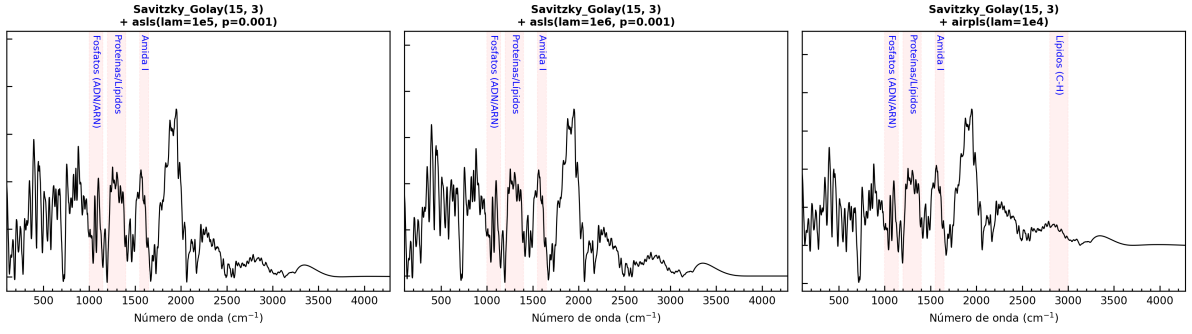


Figura 12: Galería Raman (Parte 4).

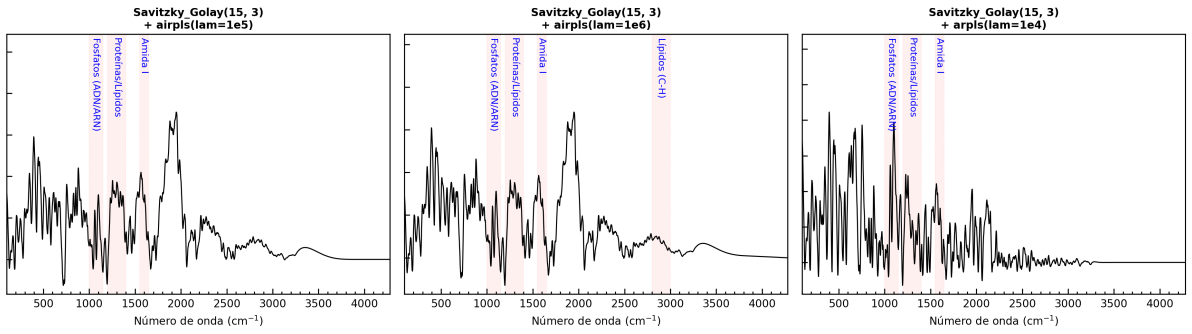


Figura 13: Galería Raman (Parte 5).

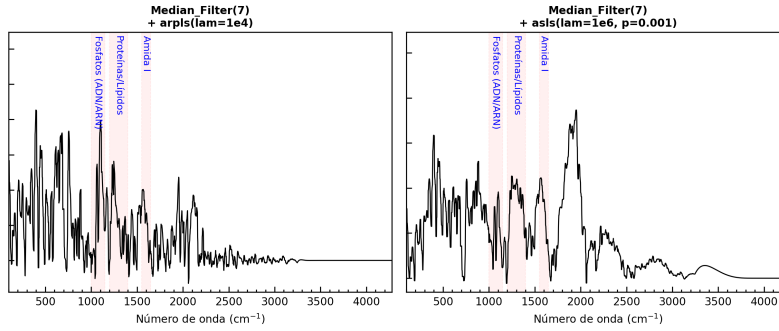


Figura 14: Galería Raman (Parte 6, mostrando las últimas 2 combinaciones y 1 celda vacía).

3.9. Comparativa Tecnológica y Criterio Clínico

La versión 0.0.3 presenta ventajas operacionales críticas frente a la red neuronal convolucional (CNN) profunda de Erzina et al. [4].

3.9.1. Tabla Comparativa de Flujos de Trabajo

La tabla 1 resume los contrastes de diseño e implicaciones clínicas:

Tabla 1: Comparación entre el flujo de Erzina et al. (CNN) y la biblioteca v0.0.3.

Criterio Metodológico	Estudio de Referencia (CNN) [4]	Biblioteca v0.0.3 (SERS)	Implicación Clínica / Oncológica
Hardware	Requiere GPU dedicada (NVIDIA CUDA) y almacenamiento masivo para frameworks.	Ejecución instantánea en CPU estándar (computadora de escritorio o laptop de laboratorio).	Alta viabilidad local: No requiere servidores en la nube ni hardware de alto coste.
Software	TensorFlow, Keras y controladores propietarios propensos a incompatibilidades.	NumPy y SciPy puros, instalable mediante zip de forma limpia y liviana.	Despliegue robusto: Instalación sin riesgos de desconfiguración en el laboratorio.
Entrada	Exige la adquisición simultánea de un triplete de espectros por paciente.	Clasificación y preprocesamiento sobre espectros individuales.	Flexibilidad analítica: Si una medición se daña, no se descarta toda la muestra.
Filtro CCD	Sin tratamiento de rayos cósmicos (delegado a la tolerancia intrínseca de la red).	Filtro determinista adaptativo MAD Z-Score e interpolación lineal de vecinos.	Seguridad clínica: Eliminación activa de artefactos físicos de radiación ionizante.
Explicabilidad	Redes neuronales profundas que actúan como cajas negras no interpretables.	Clasificador Kernel Ridge dual con visualización y proyección en espacio PCA.	Garantía médica: El clínico puede contrastar y auditar la posición del espectro frente a controles.

3.9.2. Beneficios Clave en Oncología Clínica

- Cero Latencia y Eficiencia Computacional:** El clasificador Kernel Ridge posee una solución matemática dual cerrada directa mediante álgebra lineal $((K + \lambda I)^{-1}Y)$. El entrenamiento del dataset completo toma menos de un segundo en CPU ordinaria y la predicción tarda milisegundos.
- Mayor Resiliencia al Sobreajuste (Overfitting):** La regularización de Tikhonov ($\lambda = 0,01$) penaliza los coeficientes grandes, forzando una frontera de decisión suave que generaliza con alta precisión en datos nuevos y datasets pequeños de células raras.

3. **Explicabilidad y Confianza Clínica (Auditoría Médica):** La combinación de proyección PCA y clasificación Kernel Ridge permite trazar el clúster espacial de la muestra y correlacionarlo con bandas moleculares conocidas (e.g., ácidos nucleicos a $\sim 1080 \text{ cm}^{-1}$), posibilitando al oncólogo validar de forma química el diagnóstico automático antes de tomar decisiones críticas.

4. Caso de Aplicación III: Automatización de muestreo contra bases de datos FTIR

El análisis espectroscópico de muestras complejas sintetizadas en laboratorio o adquiridas en el campo se beneficia enormemente de la automatización del preprocesamiento y el mapeo de bandas contra bases de datos de referencia. En esta sección se presenta la evaluación del núcleo computacional de la biblioteca sobre dos conjuntos de datos FTIR externos de alta relevancia en nanotecnología y biomateriales:

1. **Dataset de Óxido de Grafeno (GO) a diferentes grados de oxidación:** Compuesto por archivos ASCII (.ASC) exportados por espectrómetros PerkinElmer, conteniendo lecturas de transmitancia en función del número de onda, con codificación de texto Latin-1 y metadatos de cabecera.
2. **Dataset de Hidroxiapatita (HAp) sintetizada por método hidrotermal:** Compuesto por archivos CSV conteniendo lecturas de absorbancia en función de la longitud de onda física (convertida a número de onda) tras su adquisición.

Para evaluar la versatilidad de la biblioteca sin introducir sesgos artificiales ni forzar la semejanza de los espectros, se aplicó la búsqueda en cuadrícula combinatoria estándar (grid search) de 180 configuraciones sobre cada compuesto. A continuación, se presenta para cada muestra el script de carga y procesamiento, seguido de la visualización de la mejor configuración en espacio de Transmitancia (marcado en cajas) y en espacio de Absorbancia (marcado en cajas).

```
# Carga y analisis de GO_2G
from ftir_library import (
    load_data, run_analysis,
    plot_best, FTIR
)
x, y = load_data("GO_2G.ASC", FTIR)
res = run_analysis(x, y)
plot_best(res, "best_GO_2G.png")
```

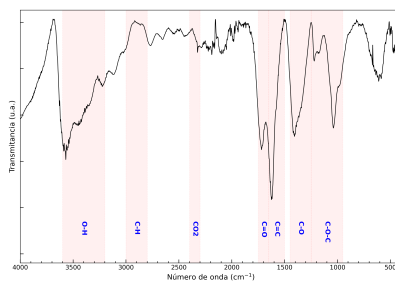


Figura 15: Transmitancia (GO 2G)

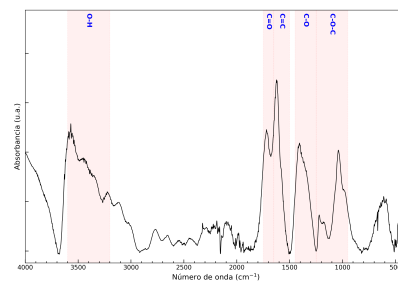


Figura 16: Absorbancia (GO 2G)

Cita: Paper: Sardinha, A. F., et al. (2020) [6] — Dataset: "FTIR spectra of Graphene oxide" [7] (DOI: 10.17632/4j2hxyrcf6.1)

```
# Carga y analisis de GO_2.5G
from ftir_library import (
    load_data, run_analysis,
    plot_best, FTIR
)
x, y = load_data("GO_2.5G.ASC",
    FTIR)
res = run_analysis(x, y)
plot_best(res, "best_GO_2.5G.png")
```

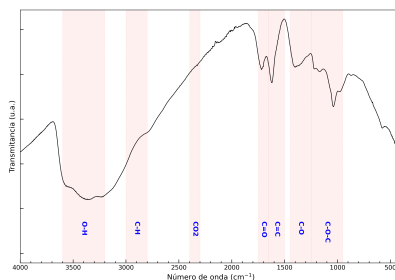


Figura 17: Transmitancia (GO 2.5G)

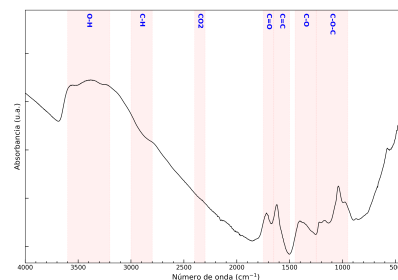


Figura 18: Absorbancia (GO 2.5G)

Cita: Paper: Sardinha, A. F., et al. (2020) [6] — Dataset: "FTIR spectra of Graphene oxide" [7] (DOI: 10.17632/4j2hxyrcf6.1)

```
# Carga y analisis de GO_3G
from ftir_library import (
    load_data, run_analysis,
    plot_best, FTIR
)
x, y = load_data("GO_3G.ASC", FTIR)
res = run_analysis(x, y)
plot_best(res, "best_GO_3G.png")
```

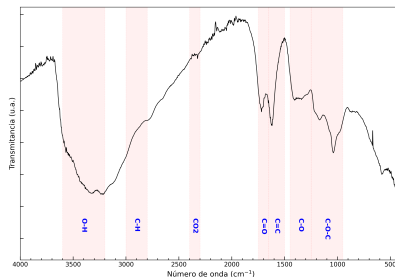


Figura 19: Transmitancia (GO 3G)

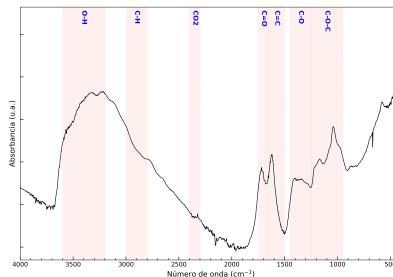


Figura 20: Absorbancia (GO 3G)

Cita: Paper: Sardinha, A. F., et al. (2020) [6] — Dataset: "FTIR spectra of Graphene oxide" [7] (DOI: 10.17632/4j2hxyrcf6.1)

```
# Carga y analisis de OG_4
from ftir_library import (
    load_data, run_analysis,
    plot_best, FTIR
)
x, y = load_data("OG_4_Original.ASC", FTIR)
res = run_analysis(x, y)
plot_best(res, "best_OG_4.png")
```

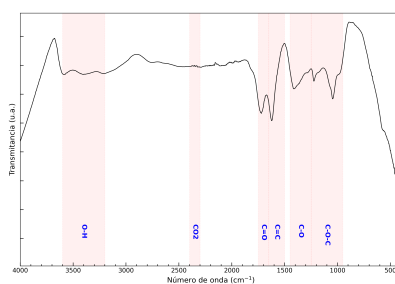


Figura 21: Transmitancia (OG 4)

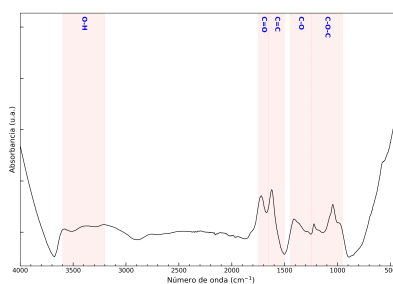


Figura 22: Absorbancia (OG 4)

Cita: Paper: Sardinha, A. F., et al. (2020) [6] — Dataset: "FTIR spectra of Graphene oxide" [7] (DOI: 10.17632/4j2hxyrcf6.1)

```
# Carga y analisis de OG_5
from ftir_library import (
    load_data, run_analysis,
    plot_best, FTIR
)
x, y = load_data("OG_5_Original.ASC", FTIR)
res = run_analysis(x, y)
plot_best(res, "best_OG_5.png")
```

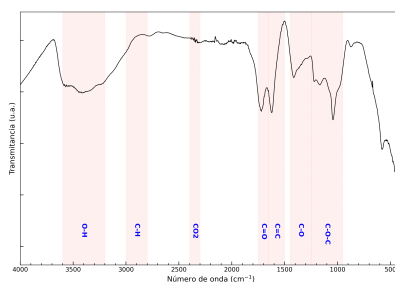


Figura 23: Transmitancia (OG 5)

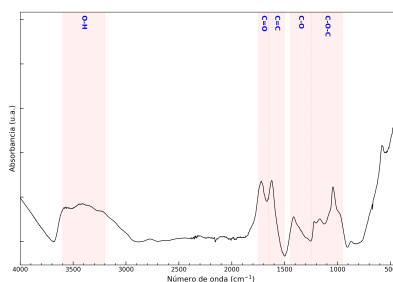


Figura 24: Absorbancia (OG 5)

Cita: Paper: Sardinha, A. F., et al. (2020) [6] — Dataset: "FTIR spectra of Graphene oxide" [7] (DOI: 10.17632/4j2hxyrcf6.1)

```
# Carga y analisis de HAp250
from ftir_library import (
    load_data, run_analysis,
    plot_best, FTIR
)
x, y = load_data("HAp250.csv", FTIR)
res = run_analysis(x, y)
plot_best(res, "best_HAp250.png")
```

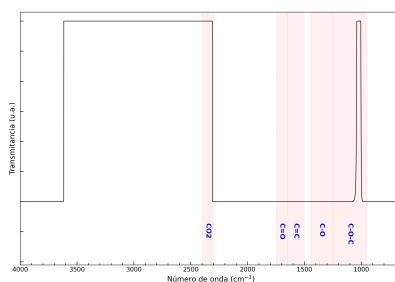


Figura 25: Transmitancia (HAp250)

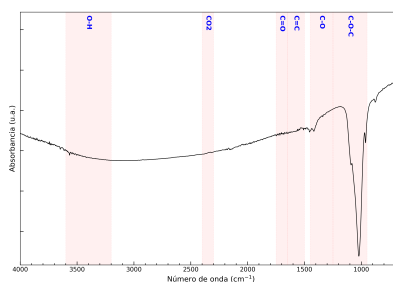


Figura 26: Absorbancia (HAp250)

Cita: Paper: Figueroa-Rosales, E. X., et al. (2023) [8] — Dataset: "FTIR & PL raw data" [9] (DOI: 10.17632/mhk9g942sr.1)

```

# Carga y analisis de HAp400
from ftir_library import (
    load_data, run_analysis,
    plot_best, FTIR
)
x, y = load_data("HAp400.csv", FTIR
)
res = run_analysis(x, y)
plot_best(res, "best_HAp400.png")

```

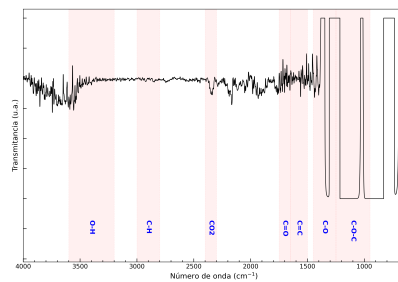


Figura 27: Transmitancia (HAp400)

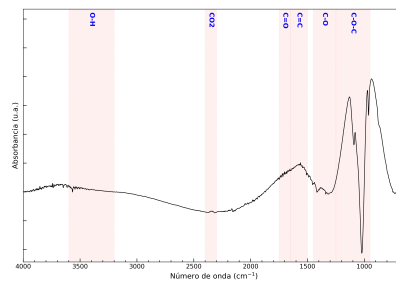


Figura 28: Absorbancia (HAp400)

Cita: Paper: Figueroa-Rosales, E. X., et al. (2023) [8] — Dataset: "FTIR & PL raw data" [9] (DOI: 10.17632/mhk9g942sr.1)

5. Manual de Desarrollo e Interfaz de Alto Nivel (API Python)

Para facilitar el uso de la biblioteca para los usuarios destinados, se implementó una interfaz de alto nivel que oculta la manipulación manual de decenas de parámetros. Esta API simplificada exporta constantes globales y funciones unificadas de auto-detección y parametrización automática:

5.1. Firmas de la API

- `x, y = load_data(filepath, format_type=None)`: Carga los datos espectroscópicos. Permite especificar manualmente el formato de la muestra mediante las constantes importadas `FTIR` o `SERS`, o por texto ("`FTIR`", "`SERS`"). Si se omite, auto-detecta según la extensión del archivo (`.csv` → `SERS`, `.txt` → `FTIR`).
- `generate_report(x, y, output_csv=None, space=None, groups_db=None)`: Genera y guarda un reporte de bandas moleculares y picos en formato CSV. Admite la especificación opcional de una base de datos de compuestos personalizada mediante `groups_db`.
- `analysis = run_analysis(x, y, space=None, groups_db=None)`: Corre el barrido combinatorio completo de 180 combinaciones y devuelve un contenedor de tipo `PipelineResult`. Admite la especificación opcional de una base de datos de compuestos personalizada mediante `groups_db`.
- `n = get_approvednumber(analysis)`: Determina dinámicamente cuántas combinaciones del grid search son consideradas óptimas o aprobadas (aquellas que logran la máxima detección de bandas analíticas de interés sin línea base RAW).
- `plot_best(analysis, output_image, mode="box", space=.absorbance)`: Grafica el espectro preprocesado óptimo marcando las bandas. Permite configurar el marcado en modo líneas ("`lines`") o cajas ("`boxes`" o "`box`"). Utiliza la base de datos de compuestos (`groups_db`) almacenada en `analysis` si está presente.
- `plot_gallery(analysis, output_prefix, max_plots=9, mode="box", space=.absorbance)`: Genera subplots horizontales 3×1 de las mejores *max_plots* configuraciones bajo el modo visual indicado. Utiliza la base de datos de compuestos (`groups_db`) almacenada en `analysis` si está presente.

5.2. Código de Ejemplo Integrado

El siguiente listado de código ejemplifica el flujo de trabajo final simplificado utilizando la biblioteca:

```
1 from ftir_library import (
2     load_data, generate_report, run_analysis,
3     plot_best, plot_gallery, get_approvednumber,
4     FTIR, SERS
5 )
6
7 # 1. CASO DE APLICACION I: ANALISIS DE MUESTRA FTIR
8 print("Analizando espectro FTIR de Grafeno...")
9 x_f, y_f = load_data("G0 coque s lav ox.txt", FTIR)
10
11 # Base de datos opcional y personalizada de compuestos y bandas
12 custom_db = {
13     "Grafeno Carbonilo (C=O)": (1680, 1750),
14     "Grafeno Aromatico (C=C)": (1550, 1630),
15     "Grafeno Epoxido (C-O-C)": (850, 1000)
16 }
17
18 generate_report(x_f, y_f, "friendly_report_FTIR.csv", groups_db=custom_db)
19 analysis_f = run_analysis(x_f, y_f, groups_db=custom_db)
20 print(f"Mejor configuracion FTIR: {analysis_f.best_config}")
21
22 # Obtener numero exacto de configuraciones exitosas
23 n_f = get_approvednumber(analysis_f)
```

```

24 print(f"Configuraciones FTIR aprobadas: {n_f}")
25
26 # Graficar mejor ajuste con lineas e imprimir galeria con cajas
27 plot_best(analysis_f, "images/friendly_best_FTIR.png", "lines")
28 plot_gallery(analysis_f, "images/friendly_gallery_FTIR", n_f, "boxes")
29
30
31 # 2. CASO DE APLICACION II: DIAGNOSTICO ONCOLOGICO RAMAN SERS
32 print("\nAnalizando espectro Raman SERS de Melanoma...")
33 x_r, y_r = load_data("Cells Raman Spectra/A/((COOH)2.csv", "SERS")
34 generate_report(x_r, y_r, "friendly_report_Raman.csv")
35 analysis_r = run_analysis(x_r, y_r)
36 print(f"Mejor configuracion Raman: {analysis_r.best_config}")
37
38 n_r = get_approvednumber(analysis_r)
39 print(f"Configuraciones Raman aprobadas: {n_r}")
40
41 # Graficar mejor ajuste con cajas e imprimir galeria con lineas
42 plot_best(analysis_r, "images/friendly_best_Raman.png", "boxes")
43 plot_gallery(analysis_r, "images/friendly_gallery_Raman", n_r, "lines")

```

Listado de código 1: Flujo de analisis utilizando constantes y numero adaptativo de galerias

5.3. Visualización de Espectros en Transmitancia y Absorbancia

La API simplificada permite graficar el espectro preprocesado óptimo tanto en el espacio de Transmitancia como en el de Absorbancia, controlándolo mediante el parámetro opcional **space**.

A continuación se presentan los ejemplos de uso y los gráficos generados para cada espacio:

```

# Graficar en Transmitancia
from ftir_library import (
    load_data, run_analysis,
    plot_best, FTIR
)
x, y = load_data("GO coque s lav ox.txt", FTIR)
res = run_analysis(x, y)
plot_best(
    res,
    "friendly_best_FTIR_trans.png",
    mode="box",
    space="transmittance"
)

```

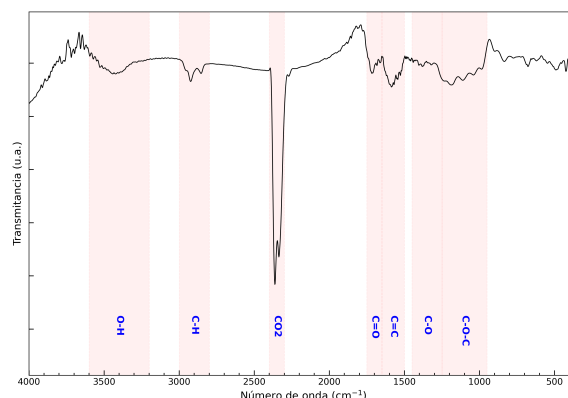


Figura 29: Muestra FTIR en Transmitancia

```

# Graficar en Absorbancia
from ftir_library import (
    load_data, run_analysis,
    plot_best, FTIR
)
x, y = load_data("GO coque s lav ox.txt", FTIR)
res = run_analysis(x, y)
plot_best(
    res,
    "friendly_best_FTIR_abs.png",
    mode="box",
    space="absorbance"
)

```

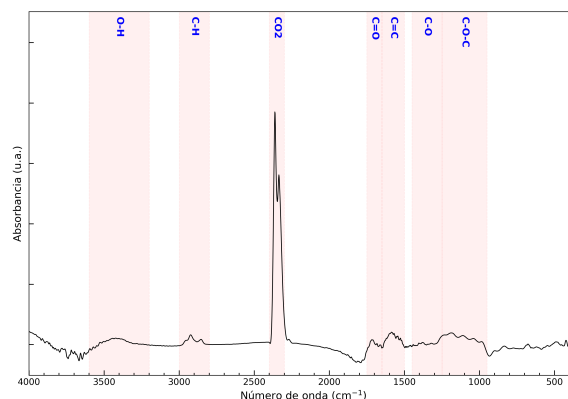


Figura 30: Muestra FTIR en Absorbancia

5.4. Soporte Local de Parámetros en Español y Buenas Prácticas

Para facilitar la adopción y accesibilidad del software por parte de investigadores y técnicos locales en laboratorios clínicos y de nanotecnología, la versión 0.0.3 de la biblioteca introduce soporte para varios parámetros en español de uso frecuente:

- **space:** Se admite "transmitancia" y ".absorbancia" como equivalentes de "transmittance" y ".absorbance".
- **mode:** Se admite "lineas" y "cajas" como equivalentes de "lines" y "box".

Sin embargo, dado que el inglés es el estándar predominante a nivel global en programación, instrumentación analítica y documentación científica, la biblioteca emite un mensaje de advertencia (**UserWarning**) para instar al usuario a familiarizarse gradualmente con los parámetros en inglés. El mensaje es el siguiente:

Advertencia: Esta biblioteca admite comandos y parámetros en español para facilitar su adopción y accesibilidad. No obstante, recomendamos familiarizarse gradualmente con la terminología en inglés, ya que constituye el estándar predominante en documentación científica, programación e instrumentación analítica.

El listado de código 2 muestra cómo se invoca de manera local y cómo se dispara el warning indicando la línea del script origen:

```
1 # Script de prueba (test1.py)
2 from ftir_library import load_data, run_analysis, plot_best, FTIR
3 x, y = load_data("G0 coque s lav ox.txt", FTIR)
4
5 # La llamada con "transmitancia" procesa los datos y emite la advertencia
6 res = run_analysis(x, y, space="transmitancia")
7
8 # La llamada con "lineas" y "transmitancia" tambien emite advertencias
9 plot_best(res, "best_G0.png", mode="lineas", space="transmitancia")
```

Listado de código 2: Uso de parametros en espanol y advertencia resultante

5.5. Configuración de Base de Datos de Compuestos Personalizada (groups_db)

La biblioteca `ftir_library` permite al usuario definir bases de datos personalizadas de compuestos y sus respectivos rangos de números de onda (wavenumbers) para el análisis automatizado y etiquetado de bandas. Esto resulta de gran utilidad cuando se trabaja con compuestos distintos al óxido de grafeno estándar o biomarcadores celulares predefinidos.

La base de datos personalizada se estructura como un diccionario en Python, donde las claves son cadenas que representan el nombre de la banda o grupo funcional, y los valores son tuplas de dos elementos conteniendo los límites inferior y superior del rango de números de onda (en cm^{-1}):

```
1 custom_db = {
2     "Grupo Hidroxilo (O-H)": (3200, 3600),
3     "Enlace Carbonilo (C=O)": (1680, 1750),
4     "Enlace Aromatico (C=C)": (1500, 1650),
5     "Estiramiento C-O-C": (950, 1200)
6 }
```

Listado de código 3: Definición de base de datos personalizada

Esta base de datos se pasa como argumento opcional `groups_db` a las funciones `run_analysis` o `generate_report`. El objeto contenedor de resultados `PipelineResult` resultante de `run_analysis` almacena automáticamente esta estructura en su atributo `groups_db`.

Al invocar los métodos de graficado `plot_best` y `plot_gallery`, la biblioteca detecta la presencia de esta base de datos dentro del objeto de análisis y dibuja los marcados de bandas (tanto en modo "lines" como "box" / "boxes") utilizando los nombres y rangos provistos por el usuario. Esto elimina la necesidad de modificar el código interno de la biblioteca para dar soporte a nuevos compuestos químicos.

6. Apéndice Algorítmico: Detalles Matemáticos y Pseudocódigos

Para garantizar la máxima transparencia científica y clínica, este apéndice técnico detalla las ecuaciones matemáticas y los pseudocódigos estructurados en español para todos los algoritmos que componen el núcleo computacional de la biblioteca `ftir_library` v0.0.3. Todos los pseudocódigos presentados adoptan de forma homogénea la indexación basada en cero (0-indexed) en coherencia con la implementación computacional de la biblioteca en Python.

6.1. Algoritmos de Suavizado y Filtros de Ruido

El suavizado se emplea para atenuar el ruido electrónico e instrumental de alta frecuencia en los espectros registrados.

6.1.1. Promedio Móvil (Moving Average)

Este filtro lineal reemplaza cada valor con la media aritmética calculada sobre una ventana local y simétrica de tamaño $W = 2k + 1$:

$$A_{\text{smooth}}[i] = \frac{1}{2k + 1} \sum_{j=-k}^k A[i + j] \quad (10)$$

Los extremos del espectro se resuelven mediante un acolchado (padding) que repite los valores de borde para evitar artefactos numéricos de truncamiento.

```
1 Algoritmo Promedio_Movil
2   Entrada: Vector y de longitud N, ventana W (impar)
3   Salida: Vector y_smooth
4
5   k = parte_entera(W / 2)
6   y_pad = Copiar y de longitud N + 2*k:
7       y_pad[0 a k-1] = y[0]
8       y_pad[k a N+k-1] = y
9       y_pad[N+k a N+2*k-1] = y[N-1]
10  y_smooth = Inicializar vector de longitud N con ceros
11  Para i desde 0 hasta N - 1 hacer:
12      Suma = 0
13      Para j desde -k hasta k hacer:
14          Suma = Suma + y_pad[i + k + j]
15      y_smooth[i] = Suma / W
16  Retornar y_smooth
17 Fin Algoritmo
```

Listado de código 4: Algoritmo de Promedio Móvil

6.1.2. Filtro de Savitzky-Golay

Realiza una regresión local por mínimos cuadrados de un polinomio de grado d sobre una ventana móvil simétrica de tamaño $W = 2k + 1$ [1]:

$$p_i(z) = \sum_{m=0}^d a_m z^m, \quad z \in [-k, k] \quad (11)$$

El valor suavizado corresponde al término independiente ajustado: $A_{\text{smooth}}[i] = p_i(0) = a_0$. Equivale matemáticamente a una convolución lineal discreta utilizando coeficientes c_j precalculados mediante la pseudoinversa de la matriz local de Vandermonde J :

$$C = (J^T J)^{-1} J^T, \quad J_{z,m} = z^m \quad (12)$$

```

1 Algoritmo Savitzky_Golay
2   Entrada: Vector y de longitud N, ventana W (impar), grado del polinomio d (
   d < W)
3   Salida: Vector y_smooth
4
5   k = parte_entera(W / 2)
6   Construir matriz J de tamaño (W x (d+1)):
7       Para cada z en [-k, k] y m en [0, d]:
8           J[z + k, m] = z^m
9   C = (J^T * J)^(-1) * J^T
10  coefs = primera fila de C
11  y_pad = Acolchar y en bordes usando simetria de espejo de longitud k
12  y_smooth = Inicializar vector de longitud N
13  Para i desde 0 hasta N - 1 hacer:
14      y_smooth[i] = Suma(coefs[j+k] * y_pad[i + j + k]) para j desde -k hasta
        k
15  Retornar y_smooth
16 Fin Algoritmo

```

Listado de código 5: Algoritmo de Savitzky-Golay

6.1.3. Filtro de Mediana (Median Filter)

Filtro no lineal que reemplaza cada punto por la mediana de la ventana móvil. Es sumamente efectivo para eliminar picos de ruido aislados de gran amplitud sin difuminar los bordes abruptos de las señales analíticas:

$$A_{\text{smooth}}[i] = \text{mediana}(\{A[i + j] \mid -k \leq j \leq k\}) \quad (13)$$

```

1 Algoritmo Filtro_Mediana
2   Entrada: Vector y de longitud N, ventana W (impar)
3   Salida: Vector y_smooth
4
5   k = parte_entera(W / 2)
6   y_smooth = Inicializar vector de longitud N
7   Para i desde 0 hasta N - 1 hacer:
8       ventana = subvector de y centrado en i de tamaño W (con relleno de
        borde)
9       Ordenar ventana de menor a mayor
10      y_smooth[i] = ventana[k]
11  Retornar y_smooth
12 Fin Algoritmo

```

Listado de código 6: Algoritmo de Filtro de Mediana

6.1.4. Filtro de Percentil (Percentile Filter)

Generaliza el filtro de mediana seleccionando el valor correspondiente a un percentil específico $P \in [0, 100]$ dentro de la ventana de vecindario local:

$$A_{\text{smooth}}[i] = \text{Percentil}_P(\{A[i + j] \mid -k \leq j \leq k\}) \quad (14)$$

```

1 Algoritmo Filtro_Percentil
2   Entrada: Vector y de longitud N, ventana W, percentil P (0 a 100)
3   Salida: Vector y_smooth
4
5   k = parte_entera(W / 2)
6   y_smooth = Inicializar vector de longitud N
7   Para i desde 0 hasta N - 1 hacer:
8       ventana = subvector de y centrado en i de tamaño W

```

```

9         Ordenar ventana de menor a mayor
10        indice_p = redondear((P / 100) * (W - 1))
11        y_smooth[i] = ventana[indice_p]
12    Retornar y_smooth
13 Fin Algoritmo

```

Listado de código 7: Algoritmo de Filtro de Percentil

6.1.5. Filtrado Adaptativo de Rayos Cósmicos (MAD Z-Score)

Las trazas CCD en espectroscopía Raman sufren interferencias por rayos cósmicos. Su eliminación adaptativa utiliza un Z-Score modificado robusto basado en la Desviación Absoluta respecto a la Mediana (MAD) de los residuos respecto a una mediana móvil previa [10]:

$$Z_i = \frac{0,6745 \cdot (d_i - \text{mediana}(d))}{\text{MAD}(d)} \quad (15)$$

donde los residuos son $d_i = y_i - y_{\text{med},i}$ y $\text{MAD}(d) = \text{mediana}(|d - \text{mediana}(d)|)$.

```

1 Algoritmo Filtrar_Rayos_Cosmicos
2   Entrada: Vector y de longitud N, ventana W, umbral theta
3   Salida: Vector y_clean
4
5   y_clean = Copiar y
6   y_med = aplicar_filtro_mediana_movil(y_clean, W)
7   d = y_clean - y_med
8   mad = mediana(abs(d - mediana(d)))
9   Si mad < 10-8 entonces mad = 10-8
10
11   z_scores = 0.6745 * (d - mediana(d)) / mad
12   spikes = buscar_indices(z_scores > theta)
13
14   Para cada idx en spikes hacer:
15       left = idx - 1
16       Mientras left este en spikes y left >= 0 hacer: left = left - 1
17       right = idx + 1
18       Mientras right este en spikes y right < N hacer: right = right + 1
19
20       Si left >= 0 y right < N entonces:
21           y_clean[idx] = y_clean[left] + (y_clean[right] - y_clean[left]) * (
22   idx - left) / (right - left)
23       Sino si left >= 0 entonces:
24           y_clean[idx] = y_clean[left]
25       Sino si right < N entonces:
26           y_clean[idx] = y_clean[right]
27
28   Retornar y_clean
29 Fin Algoritmo

```

Listado de código 8: Algoritmo de Filtrado Adaptativo de Rayos Cosmicos

6.2. Algoritmos de Corrección de Línea Base

La fluorescencia de fondo y las derivas instrumentales distorsionan el espectro. En el espacio de Absorbancia, la línea base estimada z debe aproximarse a la envolvente inferior de los espectros.

6.2.1. Ajuste Polinomial Modificado Iterativo (IModPoly)

Ajusta de forma iterativa un polinomio global de grado d . Si el espectro original en un píxel supera el polinomio estimado más la desviación estándar de los residuos σ , el dato se reemplaza por el valor

polinomial en esa iteración:

$$A^{(t)}[i] = \begin{cases} P_t(x_i) & \text{si } A[i] > P_t(x_i) + \sigma \\ A[i] & \text{de lo contrario} \end{cases} \quad (16)$$

```

1 Algoritmo IModPoly_Absorbancia
2   Entrada: Vector x, Vector y, grado d, iteraciones MaxIter
3   Salida: Linea base z
4
5   y_base = Copiar y
6   Para iter desde 1 hasta MaxIter hacer:
7       coefs = ajustar_polinomio(x, y_base, d)
8       z = evaluar(coefs, x)
9       sigma = desv_estandar(y - z)
10      Para i desde 0 hasta N - 1 hacer:
11          Si y[i] > z[i] + sigma entonces:
12              y_base[i] = z[i]
13          Sino:
14              y_base[i] = y[i]
15      coefs = ajustar_polinomio(x, y_base, d)
16      z = evaluar(coefs, x)
17      Retornar z
18 Fin Algoritmo

```

Listado de código 9: Algoritmo IModPoly (Absorbancia)

6.2.2. Mínimos Cuadrados Asimétricos (AsLS)

El algoritmo AsLS minimiza una función de coste balanceando la suavidad (derivada de segundo orden) con la fidelidad a los datos [2]:

$$S = \sum_{i=1}^N w_i (y_i - z_i)^2 + \lambda \sum_{i=3}^N (\Delta^2 z_i)^2 \quad (17)$$

donde los pesos se actualizan asimétricamente para forzar la línea base a ubicarse en el fondo:

$$w_i = \begin{cases} p & \text{si } y_i > z_i \\ 1 - p & \text{si } y_i \leq z_i \end{cases} \quad (18)$$

Se resuelve matricialmente como el sistema lineal disperso: $(W + \lambda D^T D)z = Wy$.

```

1 Algoritmo AsLS_Absorbancia
2   Entrada: Vector y de longitud N, suavizado lam, asimetria p, iteraciones
   MaxIter
3   Salida: Linea base z
4
5   D2 = construir_matriz_diferencia_segundo_orden(N)
6   D = D2^T * D2
7   w = Vector de unos de longitud N
8   Para iter desde 1 hasta MaxIter hacer:
9       W = construir_matriz_diagonal(w)
10      z = resolver_sistema_lineal(W + lam * D, w * y)
11      Para i desde 0 hasta N - 1 hacer:
12          Si y[i] > z[i] entonces:
13              w[i] = p
14          Sino:
15              w[i] = 1 - p
16      Retornar z
17 Fin Algoritmo

```

Listado de código 10: Algoritmo AsLS (Absorbancia)

6.2.3. Mínimos Cuadrados Penalizados Adaptativos (airPLS)

Modifica AsLS estimando iterativamente los pesos sin necesidad del parámetro manual de asimetría p [3]:

$$w_i = \begin{cases} 0 & \text{si } y_i \geq z_i \\ \exp\left(\frac{t \cdot (y_i - z_i)}{\text{std}(d^-)}\right) & \text{si } y_i < z_i \end{cases} \quad (19)$$

```

1 Algoritmo airPLS_Absorbancia
2   Entrada: Vector y de longitud N, suavizado lam, iteraciones MaxIter,
   tolerancia tol
3   Salida: Linea base z
4
5   D2 = construir_matriz_diferencia_segundo_orden(N)
6   D = D2^T * D2
7   w = Vector de ones de longitud N
8   Para t desde 1 hasta MaxIter hacer:
9       W = construir_matriz_diagonal(w)
10      z = resolver_sistema_lineal(W + lam * D, w * y)
11      d = y - z
12      d_neg = subvector(d < 0)
13      Si d_neg esta vacio, romper ciclo.
14      std_neg = desv_estandar(d_neg)
15      Si std_neg < tol, romper ciclo.
16      Para i desde 0 hasta N - 1 hacer:
17          Si d[i] < 0 entonces:
18              w[i] = exp(t * d[i] / std_neg)
19          Sino:
20              w[i] = 0
21      Retornar z
22 Fin Algoritmo

```

Listado de código 11: Algoritmo airPLS (Absorbancia)

6.2.4. Mínimos Cuadrados Penalizados Asimétricos Locales (arPLS)

Emplea una función logística continua asimétrica para actualizar los pesos basada en la media μ^- y desviación estándar σ^- de los residuos negativos [11]:

$$w_i = \frac{1}{1 + \exp\left(\frac{d_i - (2\sigma^- + \mu^-)}{\sigma^-}\right)} \quad (20)$$

```

1 Algoritmo arPLS_Absorbancia
2   Entrada: Vector y de longitud N, suavizado lam, iteraciones MaxIter,
   tolerancia tol
3   Salida: Linea base z
4
5   D2 = construir_matriz_diferencia_segundo_orden(N)
6   D = D2^T * D2
7   w = Vector de ones de longitud N
8   Para iter desde 1 hasta MaxIter hacer:
9       W = construir_matriz_diagonal(w)
10      z = resolver_sistema_lineal(W + lam * D, w * y)
11      d = y - z
12      d_neg = subvector(d < 0)
13      Si d_neg esta vacio:
14          mean_neg = 0.0
15          std_neg = desv_estandar(d)
16      Sino:
17          mean_neg = promedio(d_neg)
18          std_neg = desv_estandar(d_neg)

```

```

19     Si std_neg < tol, romper ciclo.
20     Para i desde 0 hasta N - 1 hacer:
21         exponente = (d[i] - (2 * std_neg + mean_neg)) / std_neg
22         exponente = limitar_entre_valores(exponente, -50, 50)
23         w[i] = 1.0 / (1.0 + exp(exponente))
24     Retornar z
25 Fin Algoritmo

```

Listado de código 12: Algoritmo arPLS (Absorbancia)

6.3. Algoritmos de Búsqueda y Optimización Paramétrica

6.3.1. Búsqueda en Cuadrícula Combinatoria (Grid Search)

Para automatizar la selección del mejor pipeline de preprocesamiento, se barren de forma cruzada todas las combinaciones paramétricas. Se calcula un Score que recompensa la detección de biomarcadores conocidos (N_g) y penaliza los picos asociados al ruido instrumental (N_{ruido}):

$$\text{Score} = N_v + N_g - w_n \cdot N_{\text{ruido}} \quad (21)$$

```

1 Algoritmo Busqueda_Cuadrícula
2   Entrada: Vector x, Vector y, Prominencia P, Distancia D, Peso w_n
3   Salida: Mejor_Config, Historial_Resultados
4
5   Historial_Resultados = Lista vacía
6   Para cada smooth_cfg in SMOOTHING_CONFIGS hacer:
7       y_smooth = aplicar_suavizado(y, smooth_cfg)
8       Para cada base_cfg in BASELINE_CONFIGS hacer:
9           Intentar:
10              z, y_corr = aplicar_linea_base(x, y_smooth, base_cfg)
11              picos_x, picos_y = detectar_picos(x, y_corr, P, D)
12              grupos_mapeados, ruido = asignar_grupos(picos_x)
13
14              score_std = calcular_score(picos_x, grupos_mapeados, ruido,
15              1.0)
16
17              score_pen = calcular_score(picos_x, grupos_mapeados, ruido, w_n
18              )
19
20              Registrar en Historial_Resultados:
21                  smooth_cfg.label, base_cfg.label,
22                  len(grupos_mapeados), len(picos_x), len(ruido), score_pen
23           Sino:
24               Ignorar configuracion fallida
25
26   Filtrar Historial_Resultados excluyendo Baseline == "RAW"
27   Mejor_Config = elemento en Historial con maximo(score_pen)
28   Retornar Mejor_Config, Historial_Resultados
29 Fin Algoritmo

```

Listado de código 13: Algoritmo Modular Combinatorio de Búsqueda en Cuadrícula

6.4. Algoritmos de Clasificación Predictiva y Machine Learning

6.4.1. Clasificación con Kernel Ridge Classifier (Dual RLS)

Resuelve de forma cerrada el clasificador no lineal de mínimos cuadrados regularizados con kernel en el espacio dual de coeficientes $\alpha_c \in \mathbb{R}^M$ para cada clase $c \in C$ [12]:

$$\alpha_c = (K + \lambda I)^{-1} Y_c, \quad K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|_2^2) \quad (22)$$

Para la clasificación multiclase se emplea el esquema One-vs-Rest (OvR). La puntuación de decisión para un espectro de prueba $x^* \in \mathbb{R}^D$ en cada clase c es:

$$f_c(x^*) = \sum_{i=1}^M \alpha_{i,c} \exp(-\gamma \|x_i - x^*\|_2^2) \quad (23)$$

y la predicción de clase $\hat{y}$ se obtiene seleccionando la clase con el máximo valor de decisión dual:

$$\hat{y} = \arg \max_{c \in C} f_c(x^*) \quad (24)$$

```

1 Algoritmo Kernel_Ridge_Multiclase_OvR
2   Entrada: Matriz X_train (M x D), Vector Y_train (M), lambda (regularizacion), gamma (ancho RBF), Clases
3   Salida: Modelos (coeficientes duales alpha para cada clase y matriz de entrenamiento)
4
5   K = calcular_matriz_kernel_rbf(X_train, X_train, gamma)
6   Modelos_Clase = diccionario_vacio()
7
8   Para cada c en Clases hacer:
9     # Codificación binaria One-vs-Rest (OvR)
10    Y_binario = Vector_de_ceros(M)
11    Para i desde 0 hasta M - 1 hacer:
12      Si Y_train[i] == c entonces:
13        Y_binario[i] = 1.0
14      Sino:
15        Y_binario[i] = -1.0
16
17    # Resolución en forma cerrada dual
18    alpha_c = resolver_sistema_lineal(K + lambda * Identidad(M), Y_binario)
19    Modelos_Clase[c] = alpha_c
20
21   Retornar Modelos_Clase
22 Fin Algoritmo

```

Listado de código 14: Entrenamiento y Predicción de Clasificador Kernel Ridge Multiclase

6.5. Algoritmos de Detección Extrema y Generación de Reportes

6.5.1. Detección Simultánea de Picos y Valles (Generación de Reportes)

Calcula los extremos de intensidad espectral local utilizando la restricción de prominencia y distancia mínima. Un pico a wavenumber x_c se define si:

$$y_c > y_{c \pm j} \quad \forall j \in [1, D], \quad \text{y} \quad \text{prominencia}(y_c) \geq P_{\text{picos}} \quad (25)$$

Y un valle en x_v satisface:

$$y_v < y_{v \pm j} \quad \forall j \in [1, D], \quad \text{y} \quad \text{prominencia}(-y_v) \geq P_{\text{valles}} \quad (26)$$

```

1 Algoritmo Generar_Reporte_Numerico
2   Entrada: Vector x, Vector y_corr, Nombre_Muestra, Ruta_CSV, P_picos, P_valles, D
3   Salida: Reporte en archivo CSV
4
5   picos_idx = buscar_maximos_locales(y_corr, prominencia=P_picos, distancia=D)
6   valles_idx = buscar_maximos_locales(-y_corr, prominencia=P_valles, distancia=D)
7

```

```

8      reporte = Lista vacia
9      grupos_mapeados, _ = asignar_grupos(x[picos_idx])
10
11     Para cada idx en picos_idx hacer:
12         w = x[idx]
13         val = y_corr[idx]
14         g = buscar_grupo_asociado(w, grupos_mapeados)
15         reporte.agregar(Tipo="Peak", Wavenumber=w, Absorbance=val, Group=g)
16
17     Para cada idx en valles_idx hacer:
18         w = x[idx]
19         val = y_corr[idx]
20         reporte.agregar(Tipo="Valley", Wavenumber=w, Absorbance=val, Group="N/A
21 ")
22
23     Ordenar reporte por Wavenumber descendente
24     Escribir_CSV(Ruta_CSV, reporte)
25 Fin Algoritmo

```

Listado de código 15: Algoritmo de Deteccion de Extremos y Generacion de Reportes

7. Apéndice Arquitectónico: Estructura de Clases y Librería

Para facilitar la mantenibilidad, escalabilidad y legibilidad de la biblioteca `ftir_library` v0.0.3, se ha diseñado una arquitectura orientada a objetos desacoplada. Este apéndice describe la estructura de clases del sistema mediante un diagrama de clases UML (Unified Modeling Language) y detalla las responsabilidades de cada componente clave.

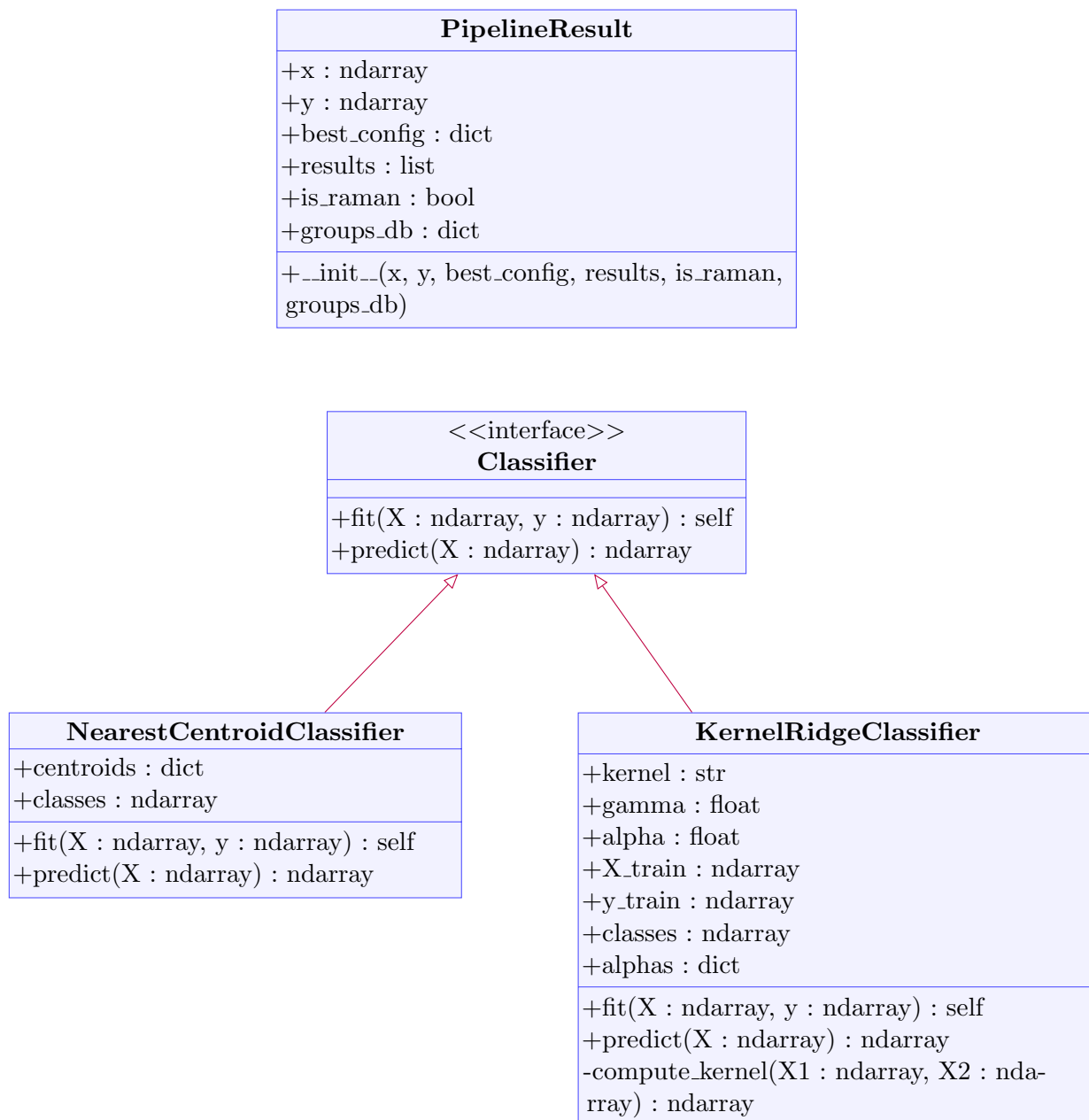
7.1. Diseño Conceptual de la Arquitectura

La biblioteca separa estrictamente las responsabilidades de preprocesamiento, búsqueda combinatoria de hiperparámetros y clasificación predictiva en tres pilares principales:

1. **Contenedor de Resultados (Capa de Abstracción de Datos):** La clase `PipelineResult` actúa como un envoltorio de alto nivel. Almacena las señales procesadas, los wavenumbers y los metadatos de la búsqueda en cuadrícula, encapsulando la complejidad de las matrices internas y permitiendo al usuario graficar y exportar resultados con facilidad.
2. **Capa de Clasificación (Machine Learning Layer):** Los algoritmos de clasificación en medio celular se implementan mediante clases que siguen una interfaz de tipo `scikit-learn` (*fit-predict-pattern*). Esto permite el polimorfismo entre clasificadores lineales basados en distancias geométricas (`NearestCentroidClassifier`) y clasificadores no lineales regularizados de alto desempeño (`KernelRidgeClassifier`).

7.2. Diagrama de Clases UML

El siguiente diagrama detalla los atributos y métodos públicos y privados de cada clase, así como las relaciones de herencia y dependencia funcional en la biblioteca:



7.3. Descripción de Módulos y Clases

7.3.1. PipelineResult

Clase unificada que sirve como interfaz entre el núcleo de preprocesamiento y el usuario de scripting.

- **x e y**: Representan los vectores o matrices de wavenumbers e intensidades espectrales preprocesados bajo la mejor configuración encontrada.
- **best_config**: Diccionario conteniendo los hiperparámetros óptimos seleccionados por la búsqueda combinatoria.
- **results**: Historial completo del score de cada combinación en la cuadrícula para permitir análisis comparativos externos.
- **groups_db**: Diccionario que almacena la base de datos de compuestos y bandas espectrales a detectar, propagada automáticamente a los métodos de visualización.

7.3.2. NearestCentroidClassifier

Clasificador lineal simple de bajo coste computacional que asigna a cada espectro de prueba la etiqueta del centroide más cercano en espacio Euclidiano. Es ideal para propósitos de benchmarking y control de

calidad.

7.3.3. KernelRidgeClassifier

Clasificador no lineal de alta precisión diagnóstica basado en mínimos cuadrados regularizados (Ridge Regression) en espacio dual con soporte para kernel Gaussiano (RBF). Permite modelar fronteras de decisión complejas y curvas no lineales de fluorescencia en espectroscopía Raman SERS. Su implementación evita el uso de dependencias pesadas utilizando álgebra lineal determinista de NumPy.

8. Conclusiones

La biblioteca `ftir_library` v0.0.3 consolida un entorno unificado de caracterización y diagnóstico de espectros vibracionales. La simplificación de su API mediante funciones que automatizan las correcciones y grid searches minimiza el error de uso en laboratorios químicos y oncológicos, mientras que el clasificador Kernel Ridge con kernel RBF aporta una precisión binaria robusta del 99.58 % para la detección de melanoma.

Referencias

- [1] Savitzky, A., & Golay, M. J. E. (1964). Smoothing and Differentiation of Data by Simplified Least Squares Procedures. *Analytical Chemistry*, 36(8), 1627-1639.
- [2] Eilers, P. H. C. (2003). A Perfect Smoother. *Analytical Chemistry*, 75(14), 3631-3636.
- [3] Zhang, Z.-M., Chen, S., & Liang, Y.-Z. (2010). An intelligent background correction method for vibrational spectroscopy. *Analyst*, 135(9), 2338-2346.
- [4] Erzina, M., Trelin, A., Guselnikova, O., Dvorankova, B., Strnadova, K., Perminova, A., Ulbrich, P., Mareš, D., Jerabek, V., Elashnikov, R., Svorcik, V., & Lyutakov, O. (2020). Precise cancer detection via the combination of functionalized SERS surfaces and convolutional neural network with independent inputs. *Sensors and Actuators B: Chemical*, 308, 127660.
- [5] Trelin, A. (2021). Cells Raman Spectra". *Kaggle Dataset*. URL: <https://www.kaggle.com/datasets/andriitrelin/cells-raman-spectra>
- [6] Sardinha, A. F., Almeida, D. A. L., & Ferreira, N. G. (2020). Electrochemical impedance spectroscopy correlation among graphene oxide/carbon fibers (GO/CF) composites and GO structural parameters produced at different oxidation degrees. *Journal of Materials Research and Technology*, 9(5), 10841-10853.
- [7] Sardinha, A. F. (2020). "FTIR spectra of Graphene oxide". *Mendeley Data*, V1. DOI: 10.17632/4j2hxyrcf6.1
- [8] Figueroa-Rosales, E. X., Hernández-Cruz, D., Robles-Águila, M. J., Rodríguez-García, M. E., & Martínez-Juárez, J. (2023). Stability of the Photoluminescent Response on Hydroxyapatite/Multi-Walled Carbon Nanotube Composites. *Photonics*, 10(8), 919.
- [9] Figueroa-Rosales, E. X. (2023). "FTIR & PL raw data". *Mendeley Data*, V1. DOI: 10.17632/mhk9g942sr.1
- [10] Whitaker, D. A., & Hayes, K. (2018). A simple algorithm for cosmic ray spike removal in Raman spectra. *Chemometrics and Intelligent Laboratory Systems*, 179, 82-84.
- [11] Baek, S.-J., Park, A., Shen, Y.-J., & Lim, J. (2015). A simple and effective baseline correction method for Raman spectra. *Analyst*, 140(1), 250-257.
- [12] Hoerl, A. E., & Kennard, R. W. (1970). Ridge Regression: Biased Estimation for Nonorthogonal Problems. *Technometrics*, 12(1), 55-67.